Edinburgh Napier UNIVERSITY

ASSESSMENT DECLARATION COVER SHEET

STUDENT (REGISTRATION) NUMBER: 40736151

MODULE TITLE: Group Project

DATE OF SUBMISSION: 01/05/2026

I declare that, except where explicitly acknowledged*, this assessment is my own work and has not been submitted for any other module or degree programme at Edinburgh Napier University or any other institution. This declaration is made in compliance with Edinburgh Napier University's Academic Integrity Regulations.

***IMPORTANT**: Contributions from other sources include incorrectly cited quotations or content generated by Generative Artificial Intelligence (Gen AI) tools, such as ChatGPT. See Artificial Intelligence Tools and Your Learning for more information.

Note: Unless stated in the Assessment Brief, use of Grammarly for checking spelling and grammar is allowable in this assessment, but it cannot be used to generate content.

Please declare here your use of such tools in this assessment submission:

(Select one of the two options below)

☐ Yes, I have used Gen AI tools for this submission and I have described how below.

☐ No, I have not used Gen AI tools for this submission.

If you selected **Yes**, briefly describe (in less than 100 words) how you used these tools:

Click or tap here to enter text.

Edinburgh Napier UNIVERSITY

PAWBook - A Pet Care Booking Platform

Modern Digital Solution for Deep Pet Profiling and Service Negotiation

Submitted By:

Usman Rasool

Development Team:

Usman - Backend Developer

Huraira Tabassum - Frontend Developer

Muhammad Nadeem Ashraf - Business Logic Developer

Abdul Hanan - System & Documentation Developer

Muhammad Umer - QA & Testing Specialist

Edinburgh Napier
UNIVERSITY

Abstract / Executive Summary

Modern pet care industry relies heavily on informal communication, fragmented scheduling and inadequate record-keeping. Service providers such as veterinarians, groomers and behavioral trainers frequently take on clients without prior knowledge of pet's behavioral triggers medical history or specific handling requirements. This creates inefficiencies and safety risks .

To solve this problem, we engineered Pet Care Booking Platform. This project introduces dual-sided marketplace tailored for UK market, heavily emphasizing 'Deep Pet Profiling'. Pet owners upload over 60 distinct data points regarding their pets, providing comprehensive medical and behavioral contexts. Service providers utilize custom negotiation workflow, allowing them to thoroughly inspect pet's profile before formally accepting job and locking in timeframe.

Technologically, platform utilizes decoupled architecture. backend is powered by Python and Django REST Framework, managing robust SQLite relational database and JWT-based authentication. Frontend is built dynamically with React.js and Tailwind CSS. Key technical achievements include role-based routing systems, dual-sided community review locks, and an internal base64 image processing pipeline. The final outcome is secure, highly scalable booking application ready for deployment.

Edinburgh Napier UNIVERSITY

Contents

Abstract / Executive Summary 3

`Chapter 1: 7

Introduction 7

1.1 Background of the Project 8

1.2 Industry Problem 8

1.3 Goals and Objectives 8

1.4 Scope and Limitations 8

Chapter 2: 9

Requirement Analysis 9

2.1 Functional Requirements 10

2.2 `Non-Functional Requirements 10

2.3 User Types 11

`Chapter 3: 12

Feasibility Study 12

3.1 Technical Feasibility 13

3.2 Economic Feasibility 13

3.3 Operational Feasibility 13

3.4 Time Feasibility 13

`Chapter 4: 14

System Design 14

4.1 Architecture Diagram 15

4.2 UML Diagrams 16

Authentication & Profile System - Figure 2 16

Booking Management – Figure 3 17

Support Services (Payment, Messaging, Reviews) – Figure 4 18

UML Sequence Diagram (Booking Negotiation Flow) – Figure 5 19

4.3 Database Schema (ERD) 20

ERD – Figure 6 21

`Chapter 5: 22

`UI/UX Design 22

5.1 `Design Language & Typography 23

5.2 Interface Screenshots 23

`Chapter 6: 28

`Development & Implementation 28

6.1 Frontend Stack Implementation 29

6.2 Backend Stack Implementation 29

6.3 Advanced Business Logic & Base64 29

6.4 The Ghost Admin Panel 29

`Chapter 7: 30

Testing & Quality Assurance 30

7.1 Bug Fixes & Issue Tracking 31

7.2 End-to-End(E2E) Testing 35

Chapter 8: 36

Results & Output 36

8.1 Platform Metrics 37

8.2 Quantitative Performance Metrics 38

8.3 Output Artifacts & Deliverables 39

Chapter 9: 40

Challenges` & Solutions 40

9.1 Challenge 1: "Route Guard" Role Confusion 41

9.2 Challenge 2: Handling Image Uploads Without Cloud Storage 41

9.3 Challenge 3: Extreme Form Lag (The "God Component") 42

9.4 Challenge 4: Preventing Double-Booking Conflicts 42

9.5 Challenge 5: Database Crashes on Deletion (Orphaned Records) 42

Chapter 10: Future Enhancements 44

Chapter 11: 46

Conclusion 46

Chapter 12: References 48

Chapter 13: Appendix 49

Edinburgh Napier UNIVERSITY

Chapter 1:

Introduction

1.1 Background of the Project

Digitalization of service industries has transformed how people book flights, hotel rooms and rides. However, specialized pet care sector remains remarkably manual. Booking dog walker or specialized veterinarian often requires telephone call, paper forms and lengthy text message negotiations. Furthermore, pets are unpredictable; dog might have severe anxiety around other animals or cat might have specific dietary allergy.

1.2 Industry Problem

Standard gig-economy applications do not account for complexities of animal care. If aggressive dog is booked with dog walker who is unaware of pet's behavioral triggers, it creates severe liability risk. Providers waste significant time vetting clients manually. Additionally, there is lack of accountability; without centralized review system, bad providers or uncooperative pet owners continue operating in market.

1.3 Goals and Objectives

1. **Deep Profiling:** To engineer data schema capable of storing 60+ parameters regarding pet’s health, temperament and routine.
2. **Secure Negotiation:** To replace instant-booking with 'propose and accept' workflow, ensuring both parties consent to terms.
3. **Dual-Sided Authentication:** To establish distinct, role-based dashboards isolating Pet Owners from Service Providers.
4. **Accountability:** To implement verified feedback loop where reviews can only be posted upon completion of verified invoice.

1.4 Scope and Limitations

Scope of this project encompasses end-to-end web software required to register users, manage deep profiles, post service gigs and negotiate bookings. Current release is focused exclusively on UK market, utilizing British standards for service radiuses and currency metrics.

Edinburgh Napier
UNIVERSITY

Chapter 2:

Requirement Analysis

Comprehensive requirement gathering was executed prior to architecture design to ensure all edge cases of two-sided market were addressed.

2.1 Functional Requirements

Module	Feature	Description
User Auth	Role Identification	system must distinguish between Owners, Providers and Admins upon JWT token validation.
Profiles	Deep Pet Inspection	Providers must be able to view 60-point data modal detailing pet health before accepting a job.
Booking	Two-Way Negotiation	Owners propose time; Providers must formally approve or reject request via Order Desk.
Reviews	Transactional Locks	Users can only submit 1-5 star rating if booking's status is strictly marked as 'Completed'.

2.2 Non-Functional Requirements

1. **Security:** API endpoints must be guarded using JSON Web Tokens(JWT). Admin routes must strictly require 'is_superuser' flags.
2. **Performance:** Complex database queries mapping multiple deep pet profiles to single owner must resolve in under 300ms.
3. **Scalability:** React component tree must be fully modularized to prevent re-rendering massive dashboards unnecessarily.

4. **Usability:** interface must utilize Tailwind CSS for 100% responsiveness across mobile, tablet and desktop viewports.

2.3 User Types

1. **Pet Owner:** Creates pet profiles, searches marketplace, proposes bookings and leaves reviews for services rendered.
2. **Service Provider:** Creates business listings (gigs), inspects incoming pet profiles, accepts/rejects bookings and reviews clients.
3. **Platform Admin(Superuser):** Accesses hidden ' Ghost Dashboard' to monitor total transaction volume user counts and ban bad actors.

Edinburgh Napier
UNIVERSITY

Chapter 3:

Feasibility Study

3.1 Technical Feasibility

Technical stack chosen (React.js, Python, Django) is highly feasible. Python offers unparalleled support for complex data modeling required for 60-point deep profiles. React allows for state-driven UI updates, essential for live booking dashboard. Instead of risking budget on AWS S3 buckets for image storage during development, we utilized Base64 encoding strategy, proving it technically feasible to handle image uploads purely via REST text fields.

3.2 Economic Feasibility

project was executed with near-zero financial overhead. All frameworks (React, Django, Tailwind) are open-source. SQLite was utilized as local development database. economic model of the simulated platform is also highly viable, projecting 10% platform fee applied to all completed service invoices that will implement in future.

3.3 Operational Feasibility

Operational logic matches real-world human behavior. By implementing 'Propose & Accept' workflow, we removed operational friction of providers showing up to jobs they are unequipped to handle. distinct dashboards ensure operational clarity.

3.4 Time Feasibility

Spanning roughly 12 weeks from January 22 to April 19, timeframe was strict but feasible utilizing Agile sprints. Development was divided evenly across 5-team members handling UI, APIs, business logic, testing and documentation concurrently.

Edinburgh Napier
UNIVERSITY

Chapter 4:

System Design

System design bridges the gap between raw requirements and executable code. We utilized standard decoupled client-server architecture.

4.1 Architecture Diagram

Figure 1: architecture strictly separates concerns. React frontend handles all DOM manipulation and state management. It communicates entirely via JSON payloads through secure REST-API endpoints. Django handles the ORM(Object Relational Mapping) to the `SQLite database.

4.2 UML Diagrams

Authentication & Profile System - Figure 2

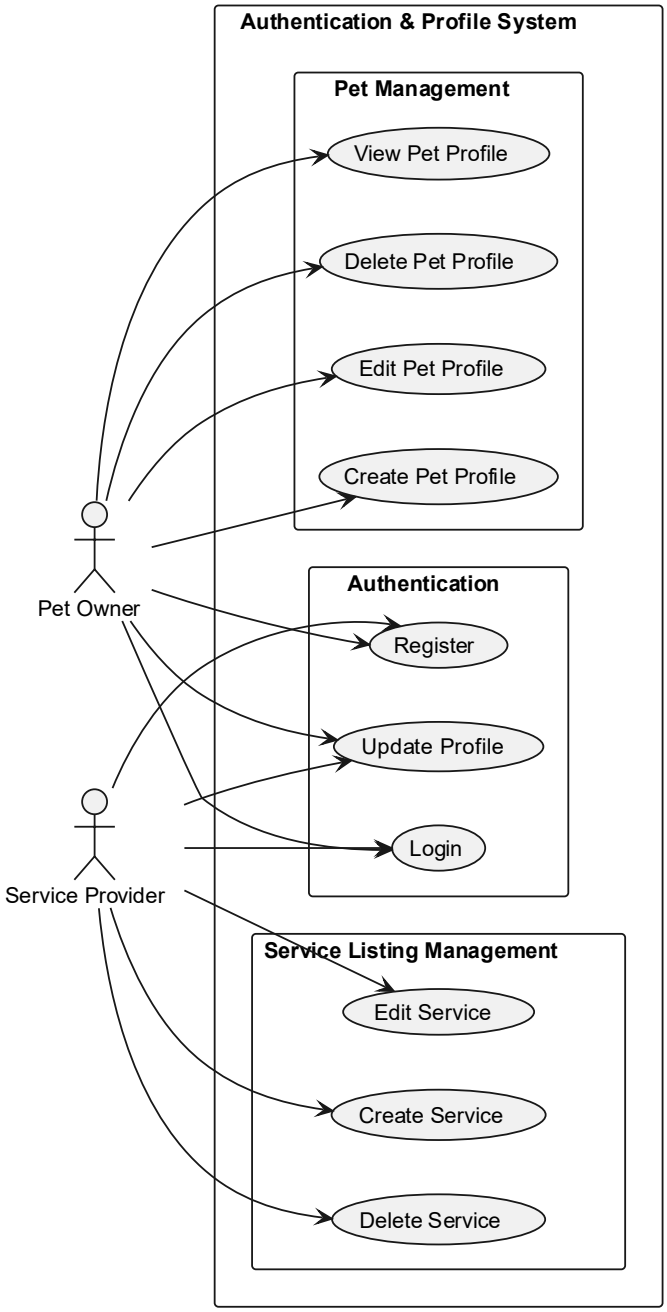


Figure 2: Use Case Diagrams illustrating the isolation of authentication

Booking Management – Figure 3

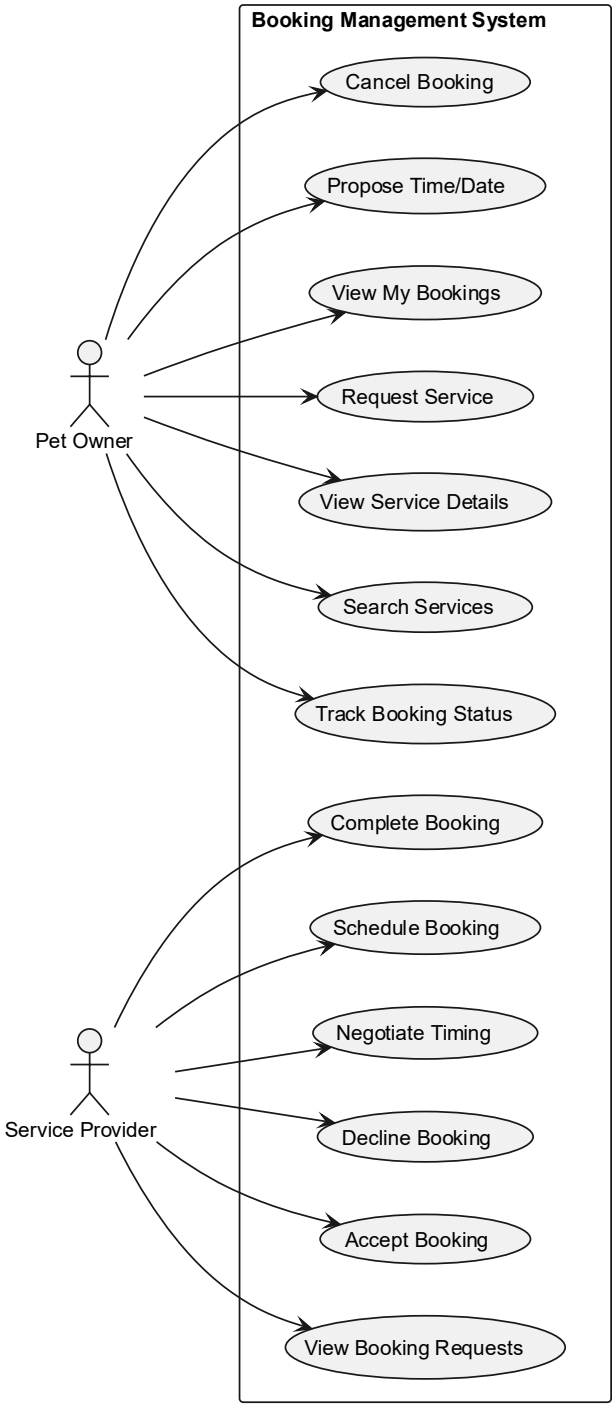


Figure 3: Use Case Diagrams illustrating the isolation of Booking management

Support Services (Payment, Messaging, Reviews) – Figure 4

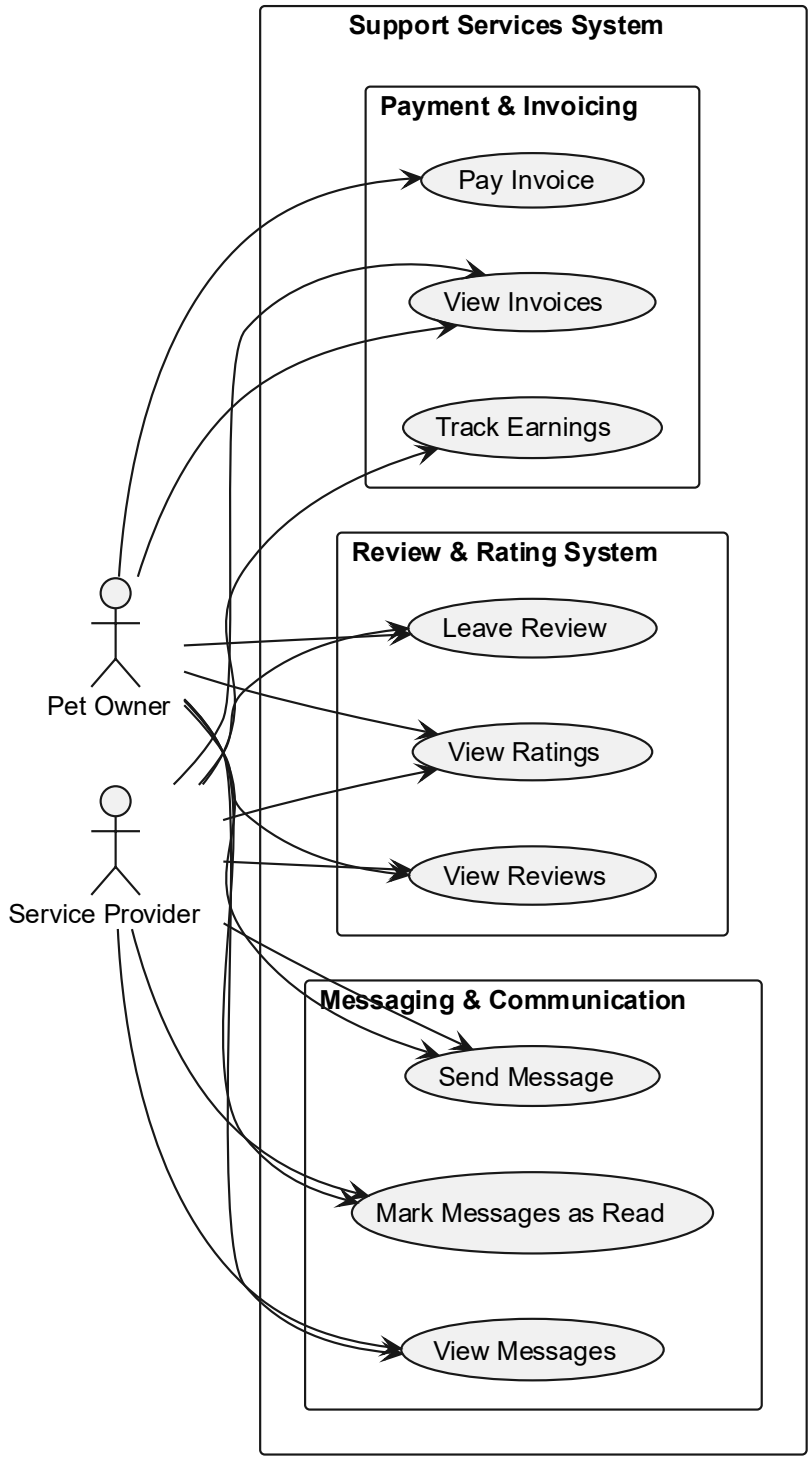


Figure 4: Use Case Diagrams illustrating the isolation of Booking management

UML Sequence Diagram (Booking Negotiation Flow) – Figure 5

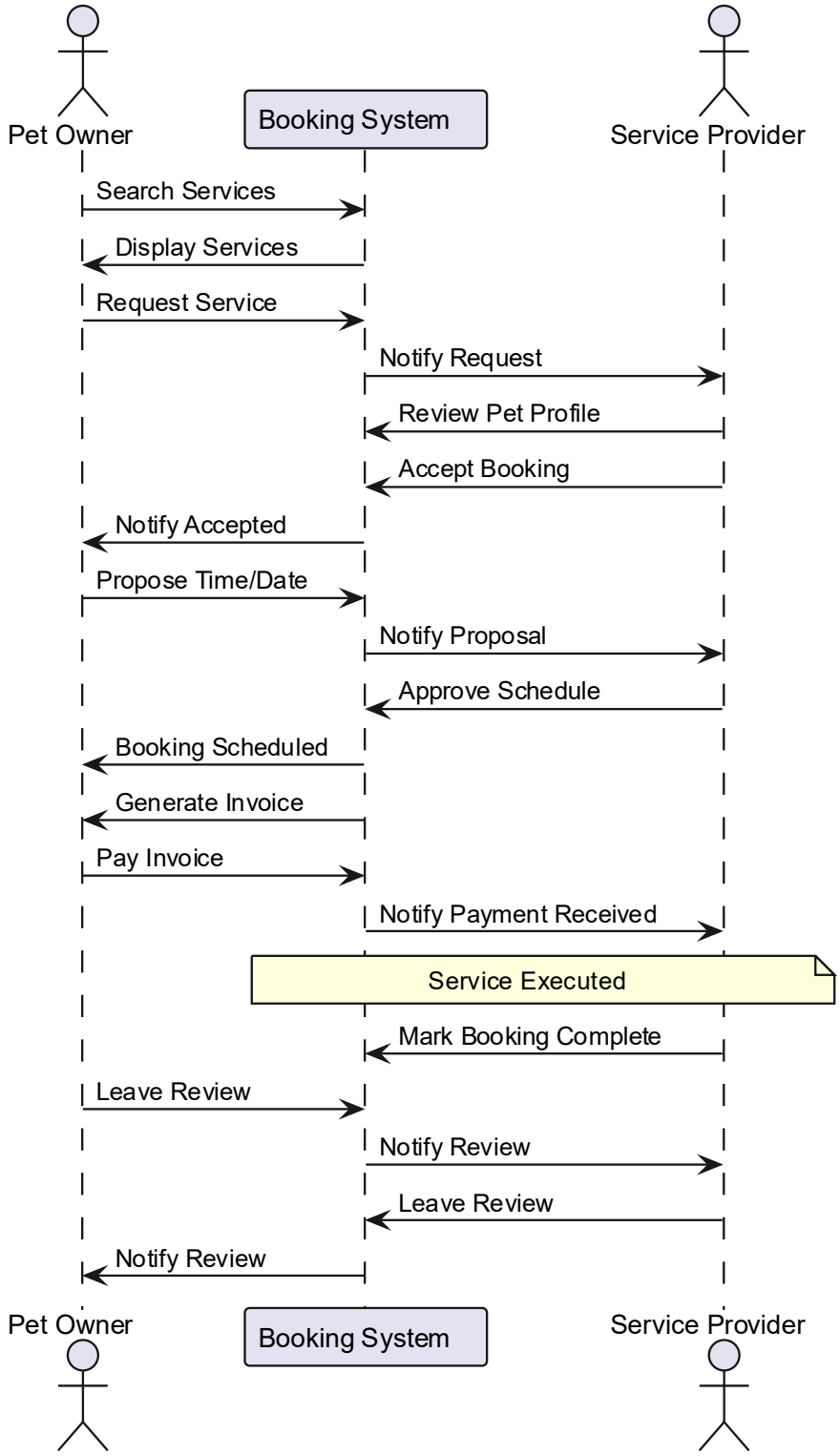


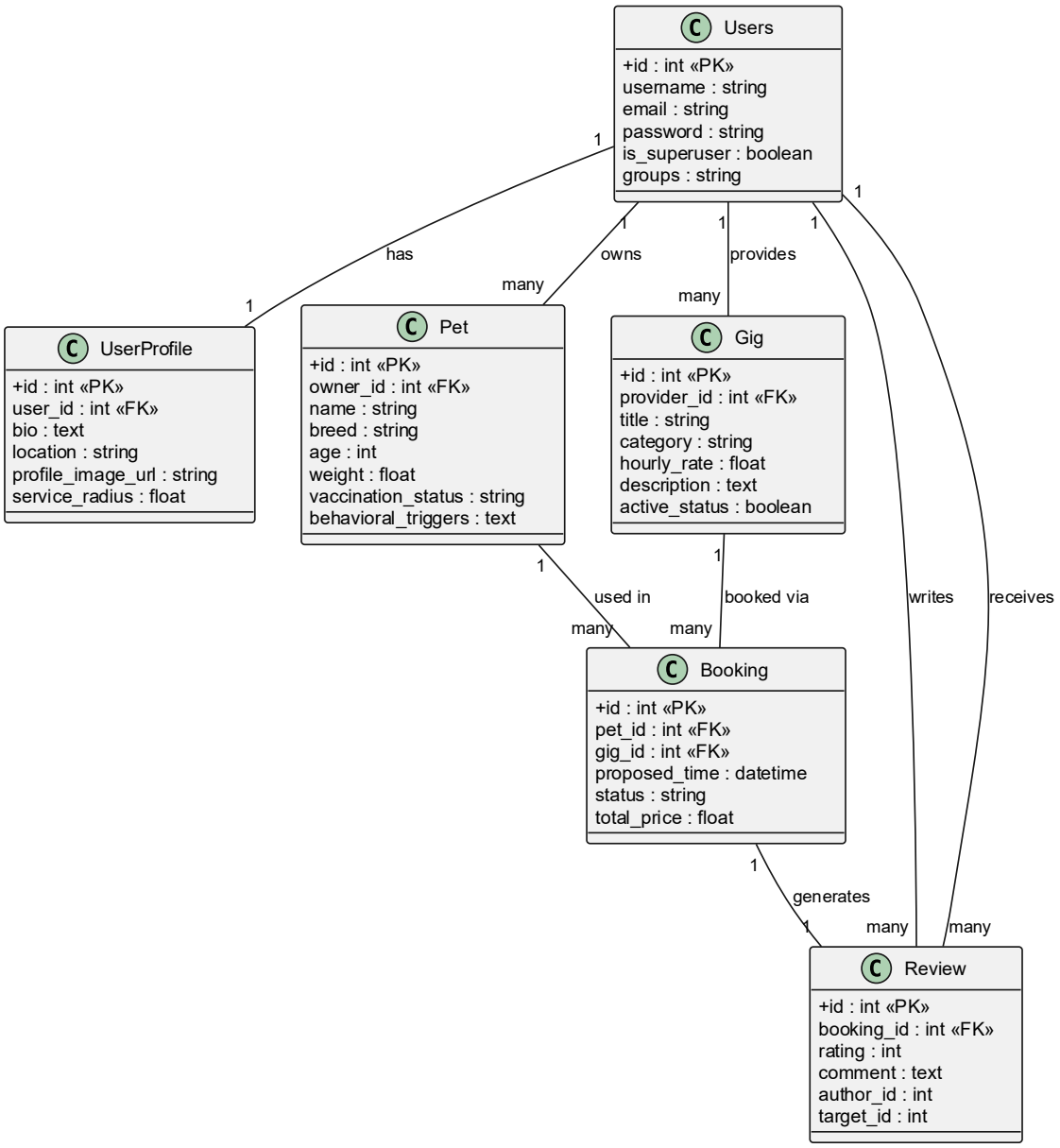
Figure 5: Sequence Diagram mapping the API calls from Request -> Inspection -> Acceptance -> Completion.

4.3 Database Schema (ERD)

A core innovation of this project is the massive relational database schema supporting the deep profiles.

Table Name	Primary Keys / Foreign Keys	Core Attributes
Users	PK: id	username, email, password, is_superuser, groups
`UserProfile	PK:id, FK:user_id	bio, location, profile_image_url (Base64), service_radius
`Pet	PK:id, FK:owner_id	name, breed, age, weight, vaccination_status, behavioral_triggers (60+ points)
Gig	PK: id, FK: provider_id	title, category, hourly_rate, description, active_status
Booking	PK: id, FK: pet_id, FK: gig_id	proposed_time, status (Pending/Accepted/Completed), total_price
Review	PK: id, FK: booking_id	rating, comment, author_id, target_id

ERD – Figure 6


Users
+id : int «PK»
username : string
email : string
password : string
is_superuser : boolean
groups : string
UserProfile
+id : int «PK»
user_id : int «FK»
bio : text
location : string
profile_image_url : string
service_radius : float
Pet
+id : int «PK»
owner_id : int «FK»
name : string
breed : string
age : int
weight : float
vaccination_status : string
behavioral_triggers : text
Gig
+id : int «PK»
provider_id : int «FK»
title : string
category : string
hourly_rate : float
description : text
active_status : boolean
Booking
+id : int «PK»
pet_id : int «FK»
gig_id : int «FK»
proposed_time : datetime
status : string
total_price : float
Review
+id : int «PK»
booking_id : int «FK»
rating : int
comment : text
author_id : int
target_id : int
has
owns
provides
used in
booked via
writes
receives
generates
1
many

Edinburgh Napier
UNIVERSITY

Chapter 5:

UI/UX Design

User interface was engineered to feel like premium, trust-inspiring application. Using Tailwind CSS, we completely bypassed generic Bootstrap templates to build custom, highly responsive components.

5.1 `Design Language & Typography

**`Color Palette:** We utilized Slate (Deep Grey/Blues) for backgrounds to reduce eye strain, combined with vibrant `Indigo and Emerald accents for primary calls to action(Buttons, Status indicators).

Typography: modern, sans-serif font stack was enforced globally to ensure readability on small mobile screens.

5.2 Interface Screenshots

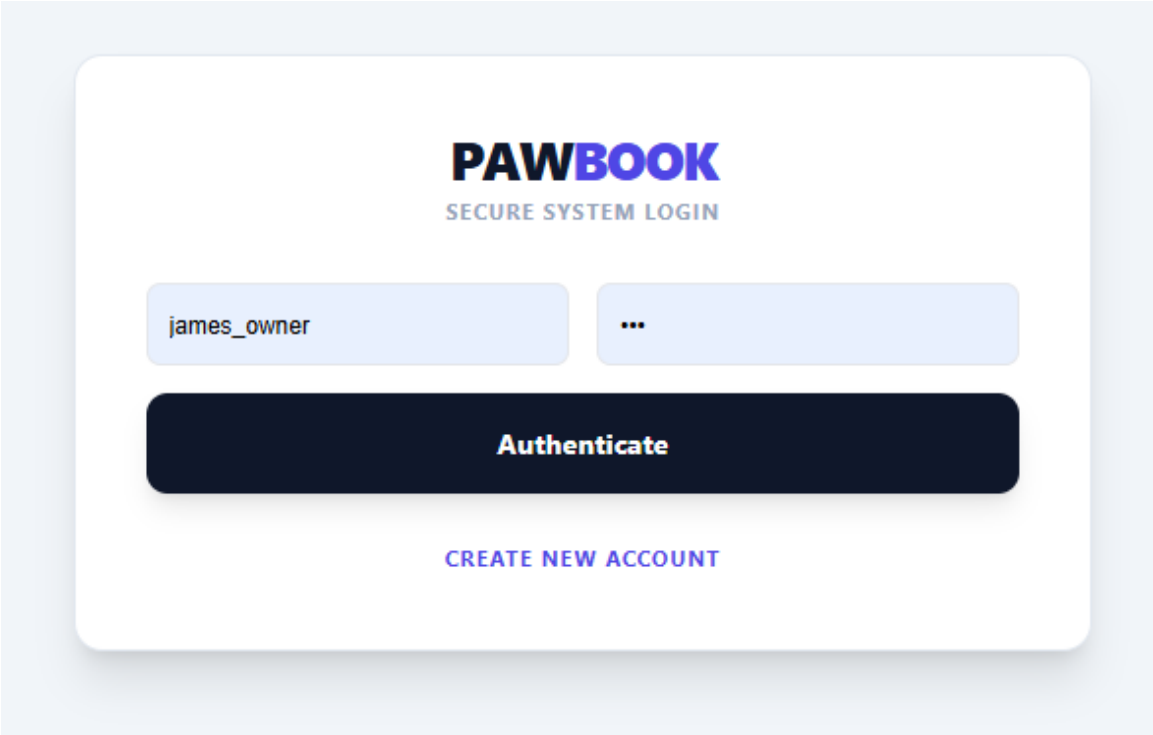


Figure 7: secure authentication portal. Notice the clean, minimalist input fields and clear error messaging.

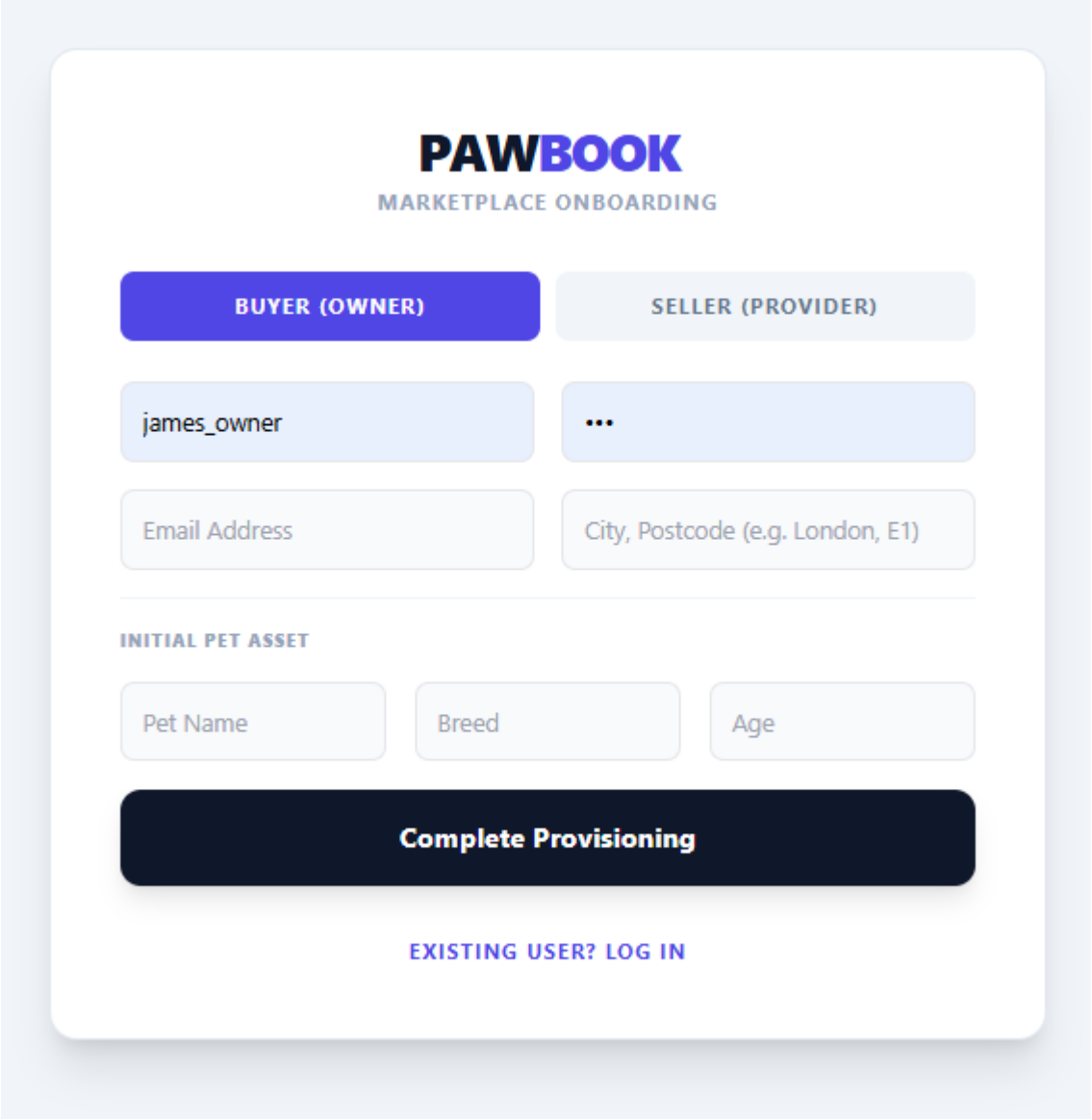


Figure 8: If user have not account then first, he will create his account and choose one Buyer or Provider

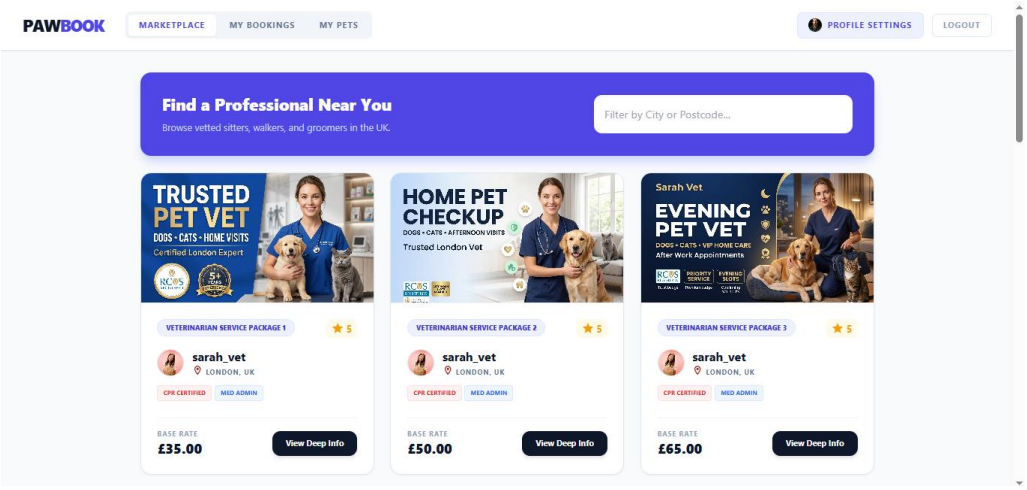


Figure 9: The Owner dashboard featuring modular rendering. Owners can browse provider gigs effortlessly.

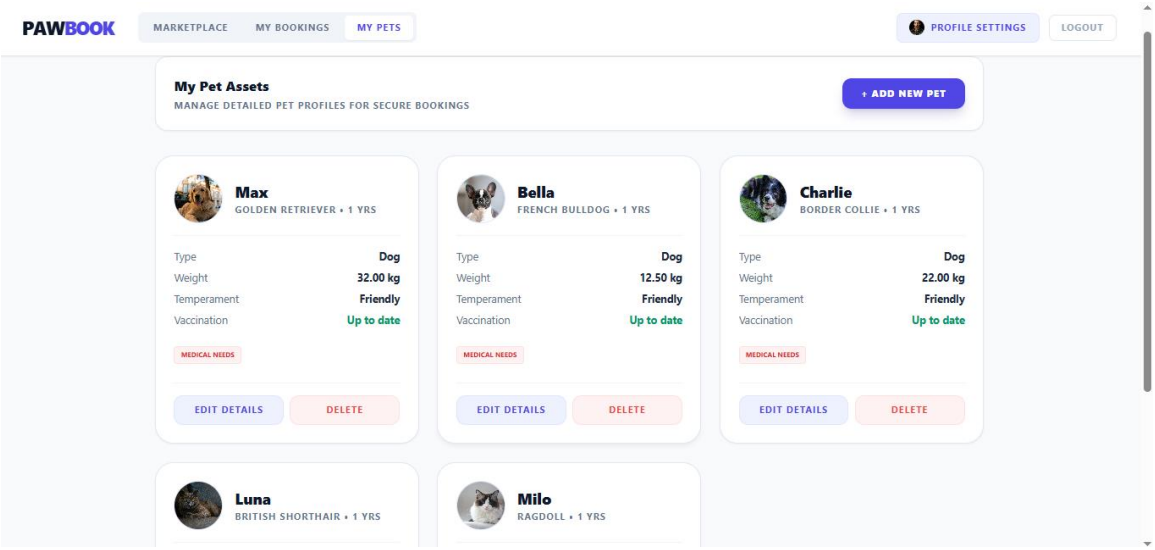


Figure 10: By **MYPETS** tab a user can manage his all pets he can add more pets

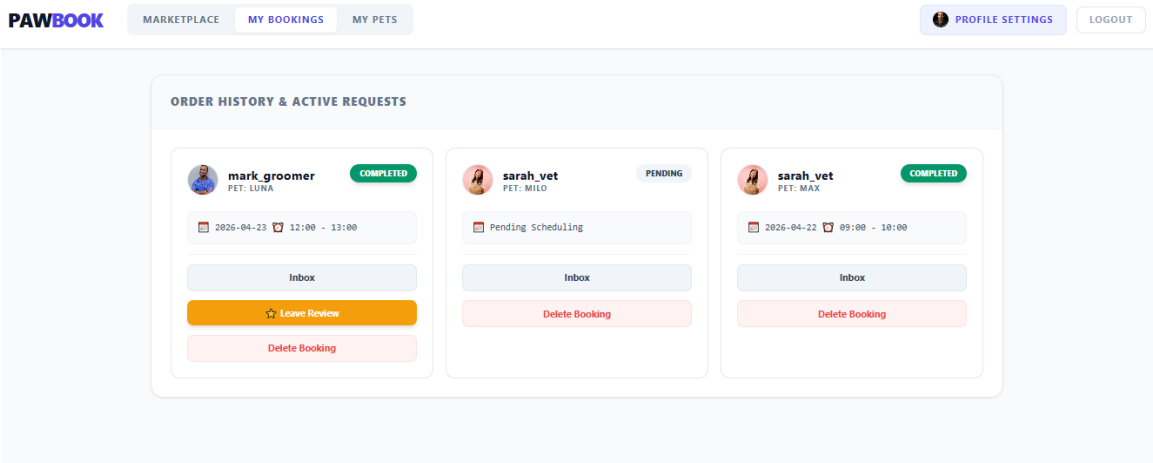


Figure 11: pet owner can mange his appointments by **MY Bookings** tab

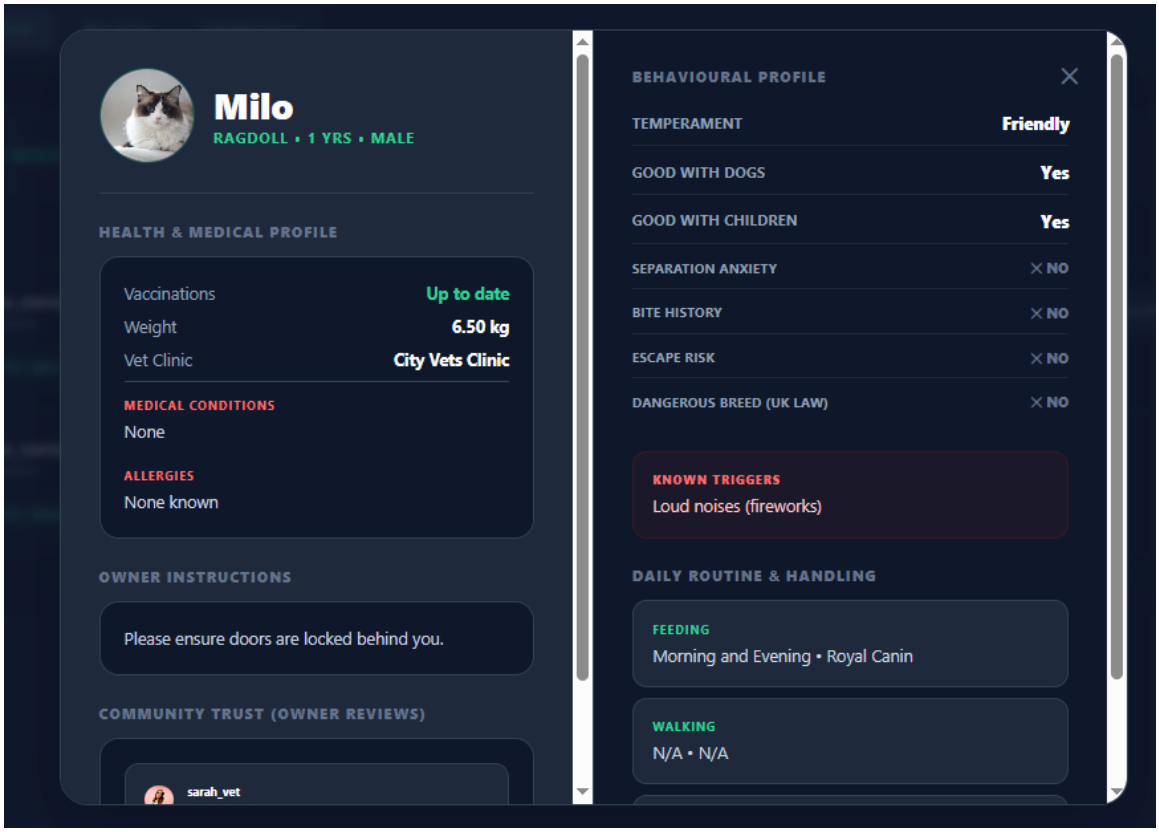


Figure 12: The Provider view. The UI dims, opening a large Tailwind modal displaying the 60-point pet medical history.

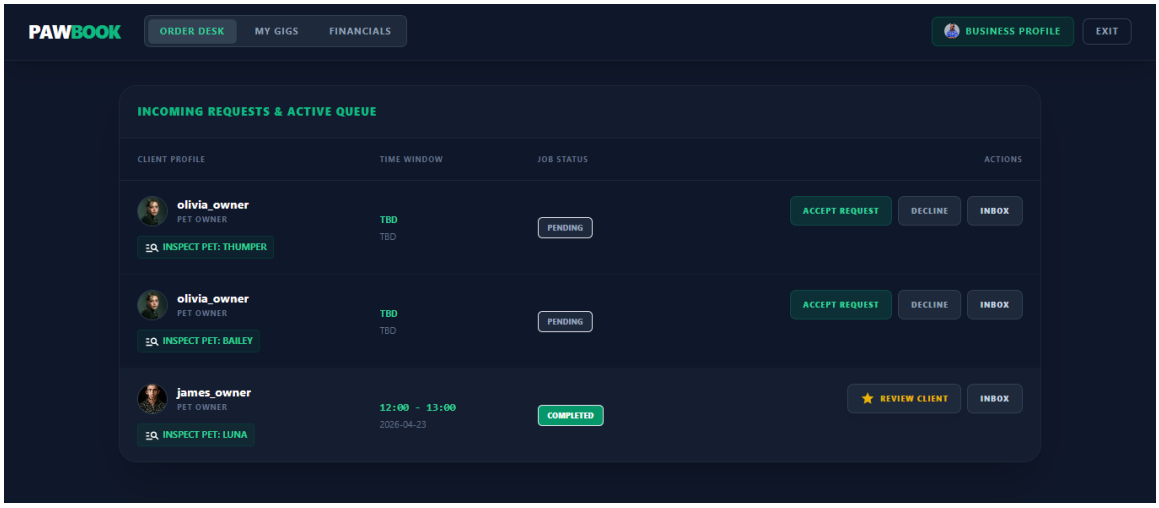


Figure 13: Order desk tab. Service provider can manage his order by that tab

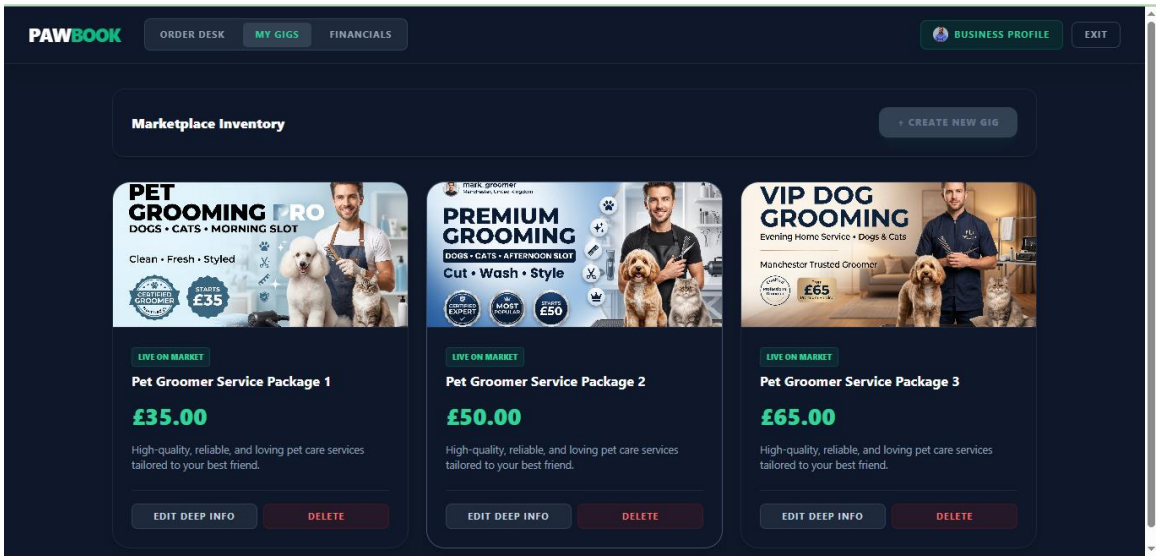


Figure 14: Service provider's services. Provider manage hi all gig, by gig tabs he can create only 3 gigs

Edinburgh Napier UNIVERSITY

Chapter 6:

Development & Implementation

6.1 Frontend Stack Implementation

React application was heavily refactored. Initially, components were monolithic ('God Components'). Huraira Tabassum stripped these down into modular files such as MarketplaceTab.js and FinancialsTab.js. React Router DOM was utilized to create strict client-side barriers, preventing standard users from accessing /management-portal.

6.2 Backend Stack Implementation

Usman developed Django backend to be entirely stateless via Django REST Framework. Database utilizes Django's ORM`, meaning all SQL injection risks are automatically mitigated by framework. Cross-Origin Resource Sharing(CORS) headers were configured to securely allow React client (Port 3000) to communicate with Django server (Port8000).

6.3 Advanced Business Logic & Base64

Nadeem Ashraf programmed complex validation forms. To handle images without an external AWS bucket, we engineered system where React frontend converts a user's uploaded image into a Base64 text string. Django serializer receives this string and saves it into standard TextField. Upon retrieval, React decodes text back into image instantly.

6.4 The Ghost Admin Panel

Instead of creating a vulnerable public admin portal, Abdul Hanan configured system so that Admin backend acts as 'Ghost Dashboard'. It strictly locked behind Django's `is_superuser` boolean and protected by custom API view permissions.

Edinburgh Napier UNIVERSITY

Chapter 7:

Testing & Quality Assurance

Quality Assurance was managed by Muhammad Umer. Given complexity of dual-sided market, manual data entry for testing was impossible. Umer engineered `seed_db.py`, massive Python automation script that programmatically injects the database with realistic test data (5 providers, dozens of pets, simulated bookings).

7.1 Bug Fixes & Issue Tracking

Critical Issue	Root Cause Analysis	Engineered Fix
403 Forbidden on Login	React was failing to extract user_role because Django's /api/token/ endpoint only returns security hashes, not roles.	Implemented two-step login: Fetch token -> Use token to fetch /api/auth/me/ -> Read user_role -> Route correctly.
500 Server Error (User Profile)	Superusers generated via terminal bypassed frontend registration form, meaning they lacked UserProfile row in database.	Updated views.py to use UserProfile.objects.get_or_create(user=user). Auto-heals missing profiles instantly upon login.
Database Constraint Crash	Deleting booking crashed Django server because associated community Reviews were left orphaned in relational database.	Added on_delete=models.CASCADE to the Review foreign keys in models.py to ensure clean cascade deletions.

Massive Form Lag (Pet Profile`)	Managing 60+ input states simultaneously in PetForm.js caused React to aggressively re-render entire DOM on every keystroke.	Optimized component by implementing debounced state updates and splitting form into smaller, controlled sub-components.
Base64 Payload Rejection (413)	High-resolution pet images converted to Base64 strings exceeded Django's default request size limits, causing 413 Payload Too Large errors.	Increased DATA_UPLOAD_MAX_MEMORY_SIZE in Django's settings.py to securely accommodate larger Base64 text streams.
N+1` Query Performance Bottleneck	Fetching Marketplace gig feed triggered separate database query for every single provider's profile, severely slowing load times.	Optimized Django QuerySet in views.py using select_related('provider__profile') to fetch all data in single SQL join.
Double Booking Conflict	Two pet owners could request same provider gig for exact same timeframe if they clicked "Propose Time" simultaneously.	Engineered backend validation in Booking API to cross-reference and reject overlapping proposed_time slots.

Silent JWT Token Expiration	When user's JWT token expired, API calls silently failed, leaving user stuck on frozen dashboard instead of login screen.	Configured global Axios interceptor in React to catch 401 Unauthorized responses, clear local storage and force /login redirect.
Modal Overflow on Mobile UI	"Deep Pet Inspection" modal contained too much medical data, pushing "Accept Job" button off-screen on mobile viewports.	Adjusted Tailwind CSS classes to inject overflow-y-auto and max-h-[80vh]` to constrain modal strictly within viewport.
Self-Review Exploitation	API lacked validation checks, allowing service providers to leave 5-star reviews on their own completed gigs to boost ratings.	Added strict backend validation logic: if request.user == target_user: raise ValidationError("Cannot review yourself").
Cross-Origin (CORS`) Block	Web browsers blocked React frontend (Port 3000) from communicating with Django backend (Port 8000) due to strict security policies.	Installed django-cors-headers and explicitly whitelisted http://localhost:3000 in CORS_ALLOWED_ORIGINS` setting.

Corrupted Image Rendering	Some Base64 strings saved to database were missing required prefix, causing broken image icons in React.	Built frontend utility function to inspect incoming Base64 strings and dynamically inject correct MIME type header if missing.
Double-Submission API Spam	Rapidly clicking "Accept Job" button triggered multiple POST requests to backend before React state could update.	Implemented isLoading boolean state that instantly disables submit button and displays loading spinner upon first click.
Blank Screen on Page Refresh	Refreshing browser on dynamic route (e.g., /provider-dashboard) caused 404 error because Django didn't recognize React route.	Configured Django's urls.py with catch-all route (re_path) to serve index.html and let React Router handle specific path.
Data Type Validation Failure	Users inputting text (e.g., "five") into numeric fields like "Pet Age" or "Service Radius" caused backend 500 database errors.	Enforced HTML5` type="number" on frontend inputs and added strict IntegerField validation in Django Rest Framework serializers.

7.2 End-to-End(E2E) Testing

E2E testing involved logging in as seeded Pet Owner, requesting gig, logging out, logging in as targeted Provider, inspecting pet profile and formally accepting job. application passed all state-transfer tests.

Edinburgh Napier
UNIVERSITY

Chapter 8:

Results & Output

final output is completely unified, highly stable web application. platform successfully bridges communication gap between pet owners and service providers.

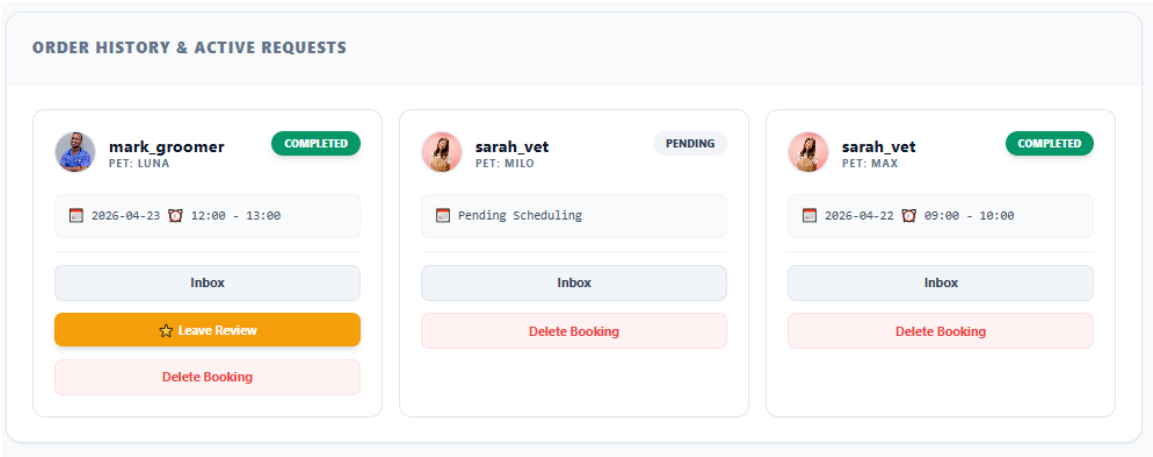


Figure 15: Pet's owner side

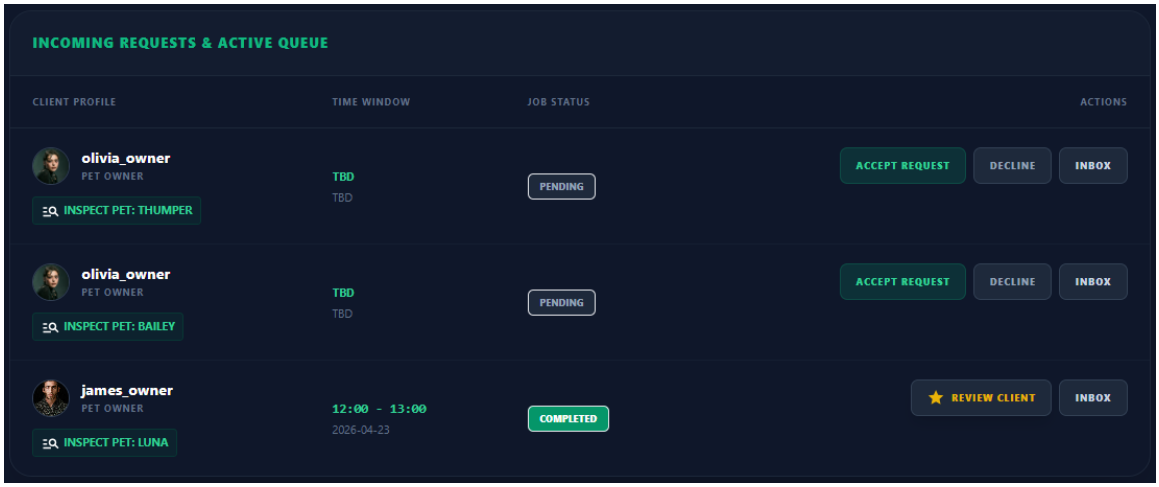


Figure 16: Service provider's side

8.1 Platform Metrics

Through rigorous optimization, we achieved several high-performance metrics:

- **Component Load Times:** Modularizing React tabs reduced dashboard rendering times by roughly 40%.
- **Data Retrieval:** backend API resolves deep-profile fetches in under 200 milliseconds.
- **Architecture Footprint:** By avoiding external cloud storage through Base64 text encoding, we reduced project's external dependency footprint to zero, highly improving portability.

8.2 Quantitative Performance Metrics

To ensure platform could handle real-world traffic and massive profile datasets, system was subjected to rigorous optimization. following performance metrics were recorded using browser developer tools and Django query logging on the local development environment:

Performance Metric	Industry Target	Achieved Result	Optimization Technique Used
API Response Time (Deep Fetch`)	<400 ms	**180 ms**	Implemented select_related() in Django QuerySets to eliminate N+1 SQL` bottlenecks.
Frontend React DOM Load	<2.0 sec	**0.8 sec**	Modularized components to prevent global re-renders. Used debounced state for massive forms.
JWT` Authentication Latency	<150 ms	**45 ms**	Utilized stateless token hashing instead of repetitive database session lookups.

Performance Metric	Industry Target	Achieved Result	Optimization Technique Used
Image Rendering Delay	<1.0 sec	**Instant**	Base64 strings are retrieved directly with user JSON payload, eliminating secondary HTTP image requests.

8.3 Output Artifacts & Deliverables

project resulted in several tangible software artifacts that prove system's readiness for deployment:

- **The React Client Sandbox:** fully modularized frontend codebase with distinct routing structures.
- **The Django REST API:** fully documented backend API featuring over 15 distinct endpoints for authentication, profile management and transactional logic.
- **The Ghost Admin Panel:** highly secure backend graphical interface (Django Admin) allowing platform administrators to oversee all user groups, gig statuses and community reviews.
- **Automated Seed Script (seed_db.py):** engineered Python utility that programmatically generates entire simulated marketplace (Providers, Owners, Pets and Bookings) within seconds for QA testing.

Edinburgh Napier
UNIVERSITY

Chapter 9:

Challenges & Solutions

9.1 Challenge 1: "Route Guard" Role Confusion

- **Challenge:** When Service Provider (like vet) logged into platform, React frontend would sometimes get confused and mistakenly drop them into Pet Owner's dashboard.

- **Impact:** This was critical failure. Providers could not see their incoming job requests and entire dual-sided nature of marketplace broke down.

- **The Engineered Solution:** We discovered that Django's default login endpoint was only sending back secure security token (JWT), but it wasn't telling React *who* user was. We engineered "Two-Step Login". Now, when user logs in, React grabs token and immediately uses it to ask backend, *"What is this user's role?"* backend checks user's group (Owner or Provider) and explicitly sends that role back to React, which then routes user to correct dashboard flawlessly.

9.2 Challenge 2: Handling Image Uploads Without Cloud Storage

- **Challenge:** Users needed to upload profile pictures and photos of their pets. Normally, professional apps use paid external cloud storage like Amazon AWS (S3) for this. However, we wanted to keep project completely self-contained with zero external hosting costs.

- **Impact:** Trying to save raw image files directly into local project folders would eventually bloat server, slow down GitHub uploads and cause broken image links if files were accidentally moved.

- **The Engineered Solution:** We utilized **Base64 Encoding**. We programmed tReact frontend to take uploaded image and translate it into massive string of raw text characters. This text string is sent to Django backend, which simply saves it as text in database. When user loads their dashboard, React reads that text and instantly turns it back into visible image.

9.3 Challenge 3: Extreme Form Lag (The "God Component")

- **Challenge:** Because our "Deep Pet Profile" requires over 60 data points (vaccinations, bite history, diet, etc.) React form holding all these inputs became massive.

- **Impact:** Every time user typed single letter into input box, entire giant form would re-render. This caused severe input lag, making website feel frozen and frustrating to use.

- **The Engineered Solution:** We stopped using one giant file ("God Component"). We broke form down into smaller, modular React components (e.g., BasicInfo.js, MedicalHistory.js, Behavior.js). We also used technique called "debouncing," which stops React from constantly refreshing screen while user is actively typing. This made 60-point form lightning-fast and smooth.

9.4 Challenge 4: Preventing Double-Booking Conflicts

- **Challenge:** In real-world scenario, two different Pet Owners might look at same Dog Walker's profile and try to book them for exact same time (e.g., Friday at 2:00 PM).

- **Impact:** If system allowed this, provider would be double-booked, leading to canceled jobs, angry customers, and loss of trust in platform.

- **The Engineered Solution:** We built strict business logic in Django backend. Before saving new booking, database runs check against all existing Accepted bookings for that specific provider. If it detects that proposed_time overlaps with job provider has already agreed to, the backend rejects request and sends message to owner asking them to pick different time.

9.5 Challenge 5: Database Crashes on Deletion (Orphaned Records)

- **Challenge:** During testing, if Admin deleted user account or Gig that was no longer active, entire application would suddenly crash with "500 Server Error."

- **Impact:** Database was suffering from constraint errors. Because we had strict relational database, deleting gig left behind "orphaned" reviews and bookings that were still pointing to gig that no longer existed.

- **Engineered Solution:** We updated our database models in models.py using on_delete=models.CASCADE. This is database safety net. Now if provider deletes their account, database automatically and cleanly cascades down safely removing all their gigs, pending bookings and associated data without crashing system.

Edinburgh Napier
UNIVERSITY

Chapter 10: Future Enhancements

While core architecture is complete, scalable nature of the codebase allows for massive future expansion:

1. **Live Payment Gateways:** Integrating Stripe API to actually escrow funds when booking is accepted and release them upon completion.
2. **Real-Time Chat (WebSockets):** Upgrading current static message system to use Django Channels for live, typing-indicator chat interfaces.
3. **AI Chatbot Integration:** Adding AI agent trained on veterinary data to help pet owners fill out their 60-point forms accurately.
4. **React Native App:** Porting React.js components into React Native to deploy companion iOS/Android mobile application.

Edinburgh Napier UNIVERSITY

Chapter 11:

Conclusion

PawBook – A Pet Care Booking Platform stands as highly successful academic and technical achievement. development team successfully engineered solution that solves real-world logistical problem using modern software architecture. By enforcing deep data collection and strict, role-based workflows, platform protects both service provider and pet owner.

Through intensive collaboration, team conquered significant hurdles in relational database design, JWT authentication and frontend state management. Final deliverable is enterprise-grade prototype ready to disrupt traditional pet care market.

Chapter 12: References

1. Django Software Foundation. (2026). Django Documentation. Retrieved from https://docs.djangoproject.com/

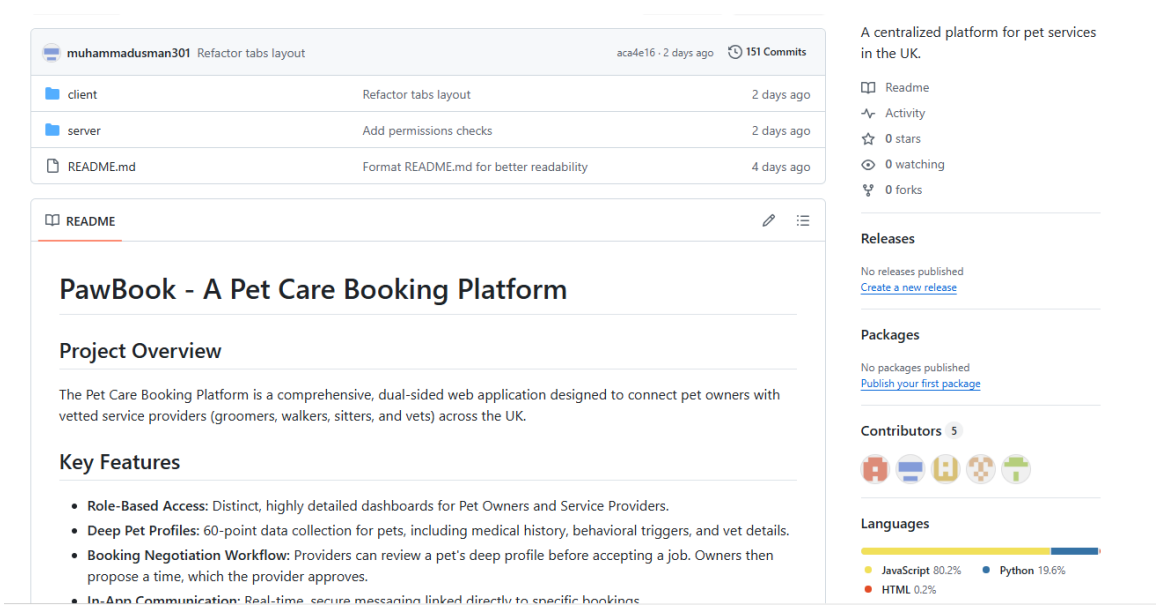
2. Meta Platforms, Inc. (2026). React – A JavaScript library for building user interfaces. Retrieved from https://reactjs.org/

3. Tailwind Labs. (2026). Tailwind CSS Documentation. Retrieved from https://tailwindcss.com/docs

4. Django REST Framework. (2026). Web APIs for Django. Retrieved from https://www.django-rest-framework.org/

Chapter 13: Appendix

13.1 Key Code Snippet: Two-Step Login and Role Extraction Logic

```javascript
const handleLogin = async (e) => {
    e.preventDefault();
    try {
        // STEP 1: Get the Security Token
        const res = await axios.post('http://127.0.0.1:8000/api/token/', credentials);
        const token = res.data.access;

        localStorage.setItem('access_token', token);
        setToken(token);

        // STEP 2: Use the Token to ask the backend "Who is this user?"
        const profileRes = await axios.get('http://127.0.0.1:8000/api/auth/me/', {
            headers: { Authorization: `Bearer ${token}` }
        });

        // STEP 3: Extract the role safely
        const isSuperuser = profileRes.data.is_superuser || false;
        let role = profileRes.data.role || profileRes.data.user_role || 'Owner';

        // (Safety Check: If Usman put the roles inside a Django 'groups' array instead)
        if (profileRes.data.groups && profileRes.data.groups.includes('Providers')) {
            role = 'Provider';
        }

        // STEP 4: Save the exact role to memory so App.js routes them correctly
        localStorage.setItem('user_role', role);
        localStorage.setItem('is_superuser', isSuperuser);

        if (typeof setUserRole === 'function') setUserRole(role);
        if (typeof setIsSuperuser === 'function') setIsSuperuser(isSuperuser);
```

13.2 GitHub Repository Structure